EFREI Paris

Développement Sécurisé – Python

Compte Rendu de Travaux Pratiques

Cybersécurité – Analyse de fichiers log SSH

Étudiant : Robert Bastien

École : EFREI Paris

Date : 03 mars 2026

Dépôt Github : https://github.com/Zeykouille/dev-secu-efrei

1. Introduction

Ce travail pratique s'inscrit dans le cadre du cours de Développement Sécurisé à l'EFREI Paris. L'objectif est de développer, en Python, un outil d'analyse de logs SSH permettant de détecter automatiquement des comportements suspects sur les serveurs d'une infrastructure réseau (DMZ, DMZ privée, routeur NAT-FW).

Les serveurs SSH sont des cibles privilégiées des attaquants : attaques automatisées, scans de comptes, tentatives de brute force… Les journaux d'authentification (/var/log/auth.log) constituent une source d'information précieuse pour détecter ces menaces, à condition de disposer d'un outil capable d'en extraire les informations pertinentes.

Ce TP est découpé en deux missions :

- Mission 1 : développement d'un analyseur de logs SSH fonctionnel, capable de détecter les tentatives de brute force, les utilisateurs invalides, les échecs d'authentification répétés, les IPs trop actives et les connexions réussies.
- Mission 2 : amélioration de cet outil pour le rendre robuste, sécurisé et exploitable en production (gestion des exceptions, lecture en streaming, validation des données, mode temps réel pour SOC).

Le fichier de log analysé est le fichier exemple fourni par l'enseignant, utilisé via le chemin /var/log/auth.log.

2. Mission 1 – Analyseur de logs SSH

2.1 Description du programme

Le programme de la Mission 1 est un script Python monolithique (ssh.py) qui lit l'intégralité du fichier de log, applique des expressions régulières pour extraire les événements pertinents, puis affiche un rapport statistique en console.

Les fonctionnalités implémentées sont les suivantes :

- Détection des échecs d'authentification par mot de passe (Failed password)
- Identification des utilisateurs invalides (Invalid user)
- Comptage des tentatives par IP et par utilisateur
- Détection des connexions SSH réussies (Accepted password / publickey)
- Détection de brute force via une fenêtre temporelle glissante
- Suivi de la première et dernière activité par IP
- Détection des événements MaxStartups / throttling SSH

2.2 Code source – ssh.py

Le code ci-dessous présente le programme complet de la Mission 1 :

ssh.py

mission-1 > ssh.py > ...

```python
#!/usr/bin/env python3

import re
from collections import defaultdict, Counter
from datetime import datetime, timedelta

LOG_FILE = "auth.log"

BRUTE_FORCE_THRESHOLD = 10              # Nombre de tentatives
BRUTE_FORCE_WINDOW_MINUTES = 2          # Fenêtre temporelle en minutes

# Extraction d'adresse IP
REGEX_IP = re.compile(r"from ([0-9]+\.[0-9]+\.[0-9]+\.[0-9]+)")

# Détection utilisateur invalide
REGEX_INVALID_USER = re.compile(r"Invalid user (\w+)")

# Détection échec authentification
REGEX_FAILED = re.compile(r"Failed password for (invalid user )?(\w+)")

# Détection connexion réussie
REGEX_SUCCESS = re.compile(r"Accepted (password|publickey) for (\w+)")

# Détection mécanisme anti-bruteforce SSH (MaxStartups / throttling)
REGEX_MAXSTARTUPS = re.compile(r"MaxStartups|throttling|drop connection")


failed_attempts = []
success_logins = []
invalid_users = Counter()
failed_by_ip = Counter()
failed_by_user = Counter()
events_by_ip = defaultdict(list)
maxstartups_events = []


def parse_timestamp(line):
    """
    Extrait le timestamp ISO présent au début de la ligne.
    Si le format n'est pas valide, retourne None.
    """
    try:
        return datetime.fromisoformat(line.split(" ")[0])
    except:
        return None
```

ssh.py

mission-1 > ssh.py > ...

```python
with open(LOG_FILE, "r", encoding="utf-8") as f:
    for line in f:

        timestamp = parse_timestamp(line)
        if not timestamp:
            continue

        ip_match = REGEX_IP.search(line)
        if ip_match:
            ip = ip_match.group(1)
            events_by_ip[ip].append(timestamp)

        failed_match = REGEX_FAILED.search(line)
        if failed_match and ip_match:
            user = failed_match.group(2)

            failed_attempts.append((timestamp, ip, user))

            failed_by_ip[ip] += 1
            failed_by_user[user] += 1


        invalid_match = REGEX_INVALID_USER.search(line)
        if invalid_match:
            invalid_users[invalid_match.group(1)] += 1

        success_match = REGEX_SUCCESS.search(line)
        if success_match and ip_match:
            user = success_match.group(2)
            success_logins.append((timestamp, ip, user))

        if REGEX_MAXSTARTUPS.search(line):
            maxstartups_events.append(line.strip())

print("\n================ GLOBAL STATS ================")
print(f"Total failed attempts: {len(failed_attempts)}")
print(f"Total successful logins: {len(success_logins)}")
print(f"Unique attacking IPs: {len(failed_by_ip)}")


print("\n================ TOP ATTACKING IPs ================")
for ip, count in failed_by_ip.most_common(10):
    print(f"{ip} -> {count} failed attempts")
```

ssh.py

mission-1 > ssh.py > ...

```python
print("\n================= TOP TARGETED USERS =================")
for user, count in failed_by_user.most_common(10):
    print(f"{user} -> {count} failed attempts")


print("\n================= INVALID USER ATTEMPTS =================")
for user, count in invalid_users.most_common(10):
    print(f"{user} -> {count} attempts")


print("\n================= SUCCESSFUL LOGINS =================")
for ts, ip, user in success_logins:
    print(f"{ts} - {user} from {ip}")


print("\n================= BRUTE FORCE DETECTION =================")

for ip, timestamps in events_by_ip.items():
    timestamps.sort()

    for i in range(len(timestamps)):
        window_start = timestamps[i]
        window_end = window_start + timedelta(minutes=BRUTE_FORCE_WINDOW_MINUTES)

        count = sum(1 for t in timestamps if window_start <= t <= window_end)
        if count >= BRUTE_FORCE_THRESHOLD:
            print(f"Possible brute force from {ip} ({count} attempts in {BRUTE_FORCE_WINDOW_MINUTES} minutes)")
            break


print("\n================= FIRST / LAST SEEN PER IP =================")
for ip, timestamps in events_by_ip.items():
    print(f"{ip} -> first: {min(timestamps)} | last: {max(timestamps)} | total events: {len(timestamps)}")

if maxstartups_events:
    print("\n================= MAXSTARTUPS / THROTTLING DETECTED =================")
    for event in maxstartups_events:
        print(event)

print("\n================= ANALYSIS COMPLETE =================")
```

2.3 Explication du code

Expressions régulières

Cinq expressions régulières sont définies pour extraire les informations clés de chaque ligne de log :

- REGEX_IP extrait l'adresse IPv4 source via le motif 'from X.X.X.X'.
- REGEX_FAILED identifie les lignes 'Failed password', en capturant le nom d'utilisateur (y compris les comptes invalides).
- REGEX_INVALID_USER détecte les tentatives sur des comptes inexistants ('Invalid user').
- REGEX_SUCCESS repère les authentifications réussies par mot de passe ou clé publique.
- REGEX_MAXSTARTUPS détecte les événements de throttling du daemon SSH (protection anti-bruteforce native).

Structures de données

Le programme utilise des structures de la bibliothèque standard Python pour stocker et agréger les données :

- Counter pour compter les occurrences par IP et par utilisateur.
- defaultdict(list) pour conserver l'historique des timestamps par IP, utilisé dans la détection de brute force.
- Des listes simples pour stocker les détails des échecs et des connexions réussies.

Détection de brute force

La détection repose sur une fenêtre temporelle glissante : pour chaque IP, on parcourt l'ensemble des timestamps et on compte le nombre d'événements survenus dans les 2 minutes suivant chaque événement. Si ce nombre dépasse le seuil de 10, l'IP est signalée comme suspecte. Cette approche permet de détecter des attaques très rapides même si elles sont étalées sur une plus longue durée.

2.4 Résultats obtenus

L'exécution du script sur le fichier de log fourni produit les résultats suivants :

Indicateur	Valeur
Total tentatives échouées	34 897
Connexions réussies	3
IPs attaquantes uniques	2

IPs les plus actives

Deux IPs sont responsables de l'ensemble des tentatives :

- 10.0.2.2 : 20 980 échecs, active de 07h54 à 15h15 (28 106 événements au total).
- 10.247.58.199 : 13 917 échecs, active de 08h20 à 10h18 (18 621 événements au total).

Utilisateurs ciblés

Le compte 'aadmin' est de loin le plus ciblé avec 34 725 tentatives, dont 5 827 sur un compte inexistant. D'autres comptes comme 'bernard', 'a', 'user', 'b', 'ndupont' et 'camille' sont également visés, avec des dizaines de tentatives chacun.

Détection de brute force

Le programme détecte clairement une attaque de brute force pour les deux IPs :

- 10.0.2.2 : 147 tentatives en moins de 2 minutes.
- 10.247.58.199 : 412 tentatives en moins de 2 minutes.

Événements MaxStartups / Throttling

Le daemon SSH a lui-même déclenché son mécanisme de protection anti-bruteforce (MaxStartups) à plusieurs reprises, notamment entre 08h14 et 08h25, avec des épisodes de throttling pour les deux IPs attaquantes. Au total, des dizaines de connexions ont été rejetées automatiquement par le serveur SSH avant même l'authentification.

Connexions légitimes

Seulement 3 connexions réussies ont été enregistrées, toutes sur des comptes valides :

- 2026-02-03 07:54 – user depuis 10.0.2.2
- 2026-02-03 08:21 – ndupont depuis 10.247.58.199
- 2026-02-03 12:18 – ndupont depuis 10.0.2.2

Ces connexions légitimes noyées dans des dizaines de milliers de tentatives illustrent bien la nécessité d'un outil d'analyse automatisé.

Exécution

```
➜ ~/dev-secu-efrei/mission-1 git:(main) X python3 ssh.py

================ GLOBAL STATS ================
Total failed attempts: 34897
Total successful logins: 3
Unique attacking IPs: 2

================ TOP ATTACKING IPs ================
10.0.2.2 -> 20980 failed attempts
10.247.58.199 -> 13917 failed attempts

================ TOP TARGETED USERS ================
aadmin -> 34725 failed attempts
bernard -> 30 failed attempts
a -> 30 failed attempts
user -> 29 failed attempts
b -> 29 failed attempts
ndupont -> 28 failed attempts
camille -> 26 failed attempts

================ INVALID USER ATTEMPTS ================
aadmin -> 5827 attempts
a -> 28 attempts
camille -> 26 attempts
b -> 23 attempts

================ SUCCESSFUL LOGINS ================
2026-02-03 07:54:25.208349+00:00 - user from 10.0.2.2
2026-02-03 08:21:56.631585+00:00 - ndupont from 10.247.58.199
2026-02-03 12:18:53.918276+00:00 - ndupont from 10.0.2.2

================ BRUTE FORCE DETECTION ================
Possible brute force from 10.0.2.2 (147 attempts in 2 minutes)
Possible brute force from 10.247.58.199 (412 attempts in 2 minutes)

================ FIRST / LAST SEEN PER IP ================
10.0.2.2 -> first: 2026-02-03 07:54:25.208349+00:00 | last: 2026-02-03 15:15:19.641284+00:00 | total events: 28106
10.247.58.199 -> first: 2026-02-03 08:20:47.053012+00:00 | last: 2026-02-03 10:18:07.598780+00:00 | total events: 18621

================ MAXSTARTUPS / THROTTLING DETECTED ================
2026-02-03T08:14:54.943437+00:00 user sshd[1621]: error: beginning MaxStartups throttling
2026-02-03T08:14:54.943491+00:00 user sshd[1621]: drop connection #14 from [10.0.2.2]:20316 on [10.0.2.15]:22 past MaxStartups
2026-02-03T08:20:46.995444+00:00 user sshd[1621]: exited MaxStartups throttling after 00:05:52, 29 connections dropped
2026-02-03T08:21:44.162650+00:00 user sshd[1621]: error: beginning MaxStartups throttling
```

3. Mission 2 – Outil professionnel et sécurisé

3.1 Architecture et améliorations apportées

La Mission 2 consiste en une refonte complète du programme de la Mission 1. Le code monolithique est décomposé en modules distincts, chacun ayant une responsabilité claire. Cette approche améliore la maintenabilité, la testabilité et la sécurité du code.

Fichier	Rôle	Amélioration principale
config.py	Configuration centralisée	Paramètres isolés et modifiables sans toucher au code
logger_config.py	Journalisation	Remplacement des print() par un logger horodaté
parser.py	Parsing et validation	Validation IP via ipaddress, gestion des lignes malformées
detector.py	Détection brute force	deque bornée (maxlen=1000) pour éviter le log flooding
realtime.py	Lecture en temps réel	Suivi du fichier log en live (tail -f), mode SOC
main.py	Point d'entrée	Gestion des exceptions (FileNotFoundError, PermissionError…)

3.2 Code source de la Mission 2

config.py

```
config.py ×
ssh_monitor > config.py > ...
1  LOG_FILE = "/var/log/auth.log"
2
3  BRUTE_FORCE_THRESHOLD = 10
4  BRUTE_FORCE_WINDOW_MINUTES = 2
5
6  IP_ACTIVITY_THRESHOLD = 50
7
8  REALTIME_MODE = False
```

logger_config.py

```
logger_config.py ×
ssh_monitor > logger_config.py > ...
1  import logging
2
3  def setup_logger():
4      logging.basicConfig(
5          level=logging.INFO,
6          format="%(asctime)s - %(levelname)s - %(message)s"
7      )
8      return logging.getLogger("ssh_monitor")
```

parser.py

parser.py

ssh_monitor > parser.py > ...

```python
import re
import ipaddress
from datetime import datetime

REGEX_IP = re.compile(r"from (\S+)")
REGEX_FAILED = re.compile(r"Failed password for (invalid user )?(\w+)")
REGEX_SUCCESS = re.compile(r"Accepted (password|publickey) for (\w+)")
REGEX_INVALID = re.compile(r"Invalid user (\w+)")

def parse_line(line):
    try:
        timestamp = datetime.now()  # simplifié ici

        ip_match = REGEX_IP.search(line)
        if not ip_match:
            return None

        raw_ip = ip_match.group(1)

        # Validation IP réelle
        try:
            ip = str(ipaddress.ip_address(raw_ip))
        except ValueError:
            return None

        if REGEX_FAILED.search(line):
            return ("failed", ip)

        if REGEX_SUCCESS.search(line):
            return ("success", ip)

        if REGEX_INVALID.search(line):
            return ("invalid", ip)

        return None

    except Exception:
        return None
```

detector.py

detector.py ×

ssh_monitor > detector.py > ...

```python
from collections import defaultdict, deque
from datetime import timedelta, datetime
from config import BRUTE_FORCE_THRESHOLD, BRUTE_FORCE_WINDOW_MINUTES

failed_by_ip = defaultdict(lambda: deque(maxlen=1000))
alerted_ips = set()

def process_event(event):
    event_type, ip = event

    if event_type == "failed":
        failed_by_ip[ip].append(datetime.now())
        check_bruteforce(ip)

def check_bruteforce(ip):
    timestamps = failed_by_ip[ip]
    now = datetime.now()

    window = timedelta(minutes=BRUTE_FORCE_WINDOW_MINUTES)

    count = sum(1 for t in timestamps if now - t <= window)

    if count >= BRUTE_FORCE_THRESHOLD:
        if ip not in alerted_ips:
            print(f"[ALERT] Possible brute force from {ip}")
            alerted_ips.add(ip)
```

realtime.py

realtime.py ×

ssh_monitor > realtime.py > ...

```python
import time

def follow(file):
    file.seek(0, 2)
    while True:
        line = file.readline()
        if not line:
            time.sleep(0.5)
            continue
        yield line
```

main.py

main.py

ssh_monitor > main.py > ...

```python
import sys
from parser import parse_line
from detector import process_event
from realtime import follow
from config import LOG_FILE, REALTIME_MODE
from logger_config import setup_logger

logger = setup_logger()

def main():
    try:
        with open(LOG_FILE, "r") as f:

            if REALTIME_MODE:
                lines = follow(f)
            else:
                lines = f

            for line in lines:
                event = parse_line(line)
                if event:
                    process_event(event)

    except FileNotFoundError:
        logger.error("Log file not found")
        sys.exit(1)

    except PermissionError:
        logger.error("Permission denied")
        sys.exit(1)

    except Exception as e:
        logger.exception("Unexpected error")
        sys.exit(1)

if __name__ == "__main__":
    main()
```

3.3 Explication des améliorations

Robustesse et gestion des exceptions (main.py)

Dans la Mission 1, l'absence du fichier de log provoquait un crash non contrôlé. La Mission 2 encadre l'ouverture du fichier dans un bloc try/except qui distingue trois cas : fichier introuvable (FileNotFoundError), accès refusé (PermissionError), et toute autre erreur inattendue (Exception). Dans chaque cas, le message d'erreur est journalisé via le logger et le programme s'arrête proprement avec sys.exit(1).

Protection contre le log flooding (detector.py)

La Mission 1 stockait tous les timestamps dans une liste non bornée. Un attaquant générant des millions de lignes de log pourrait provoquer une saturation mémoire (DoS). Dans la Mission 2, on utilise une deque avec maxlen=1000 : seuls les 1000 derniers événements par IP sont conservés en mémoire, ce qui borne la consommation mémoire quelle que soit la taille du fichier.

Validation et sécurité des données (parser.py)

Le parser de la Mission 1 supposait un format de log parfait. La Mission 2 ajoute deux niveaux de validation : le bloc try/except global dans parse_line() ignore les lignes malformées sans faire planter le programme, et la validation via le module ipaddress vérifie que l'adresse IP extraite est syntaxiquement valide avant de l'utiliser. Les adresses IP malformées ou injectées sont silencieusement ignorées.

Lecture en streaming (realtime.py)

La Mission 1 lisait le fichier ligne par ligne avec une boucle for standard, ce qui est acceptable pour un fichier statique. La Mission 2 ajoute un mode temps réel via la fonction follow(), qui se positionne en fin de fichier (seek(0,2)) et surveille l'arrivée de nouvelles lignes avec un polling de 0.5 seconde. Ce mode est activable via le flag REALTIME_MODE dans config.py, ce qui permet de basculer entre analyse statique et monitoring live sans modifier le code.

Journalisation professionnelle (logger_config.py)

Les print() de la Mission 1 sont remplacés par le module logging standard de Python. Chaque message est horodaté et catégorisé (INFO, ERROR, EXCEPTION), ce qui est indispensable pour un outil utilisé en production ou dans un SOC. Le logger est configuré une seule fois à l'initialisation et partagé entre les modules.

Centralisation de la configuration (config.py)

Tous les paramètres ajustables (chemin du fichier, seuils de détection, mode temps réel) sont regroupés dans un fichier config.py dédié. Cela évite de devoir modifier le code source pour adapter le comportement du programme, ce qui est une bonne pratique de développement sécurisé.

Résultat

```
→ ~/dev-secu-efrei git:(main) X python3 ssh_monitor/main.py
[ALERT] Possible brute force from 10.0.2.2
[ALERT] Possible brute force from 10.247.58.199
```

4. Mission 3 – Analyse SOC : logs FICOBA (DGFiP)

4.1 Contexte et objectifs

Cette mission s'inscrit dans un contexte d'analyse post-incident SOC sur les logs applicatifs du système FICOBA de la DGFiP (Direction Générale des Finances Publiques). FICOBA est le fichier national des comptes bancaires et assimilés, dont la compromission représente un risque majeur pour les citoyens et les institutions.

Suite à une cyberattaque détectée, l'objectif est de développer un outil Python permettant d'analyser les logs d'accès FICOBA afin de détecter les indicateurs suivants :

- Usurpation d'identité (connexion depuis une IP externe avec des identifiants valides)
- Extraction massive de données (volume de requêtes anormalement élevé)
- Accès hors horaires (connexions en dehors des plages 06h00-20h00)
- Contournement ou échec MFA (MFA_FAIL, MFA_BYPASS)
- Rafales de requêtes suspectes (burst de plus de 50 requêtes en moins de 10 secondes)

<u>Format des logs FICOBA</u>

Chaque entrée du fichier de log suit le format structuré suivant :

```
[TIMESTAMP] USER | ROLE | IP | APP | ACTION | RESOURCE | QUERY_COUNT | STATUS | MFA |
SESSION_ID
```

Les actions possibles sont LOGIN, LOGOUT, SEARCH et EXPORT. Le champ QUERY_COUNT représente le nombre de lignes interrogées ou exportées lors de l'action.

4.2 Analyse des activités dans les logs

Avant de présenter l'outil, voici une synthèse des trois catégories d'activité identifiées dans les logs fournis :

Activité normale

Un agent effectue 1 à 5 requêtes par session depuis une IP interne, avec MFA validé. Les sessions durent quelques minutes et les volumes restent faibles (1 à 3 IBAN ou PERSONNES consultés).

Activité légèrement anormale (p.leroy)

L'agent p.leroy effectue trois requêtes IBAN successives (25, 32 et 41 résultats) en moins de deux minutes depuis l'API FICOBA. Le volume total de 98 résultats en moins de 47 secondes constitue un premier signal d'alerte, même si le MFA est valide et l'IP interne.

Compromission avérée (m.bernard)

Le cas le plus grave concerne le compte m.bernard le 31 janvier 2026 à 22h14. Plusieurs indicateurs convergent vers une compromission caractérisée :

- Connexion depuis l'IP 185.193.44.21, externe au réseau ministériel.
- Première tentative avec MFA_FAIL, puis contournement immédiat avec MFA_BYPASS.
- Extraction de 500 personnes, 1200 IBAN et trois exports CSV totalisant 83 000 lignes en moins de 50 secondes.
- L'ensemble de l'activité suspecte se déroule en dehors des horaires bureautiques (22h14).

Indicateur	Valeur normale	Valeur observée	Verdict
Requêtes par session	1–5	83 000+	Critique
IP source	10.x / 172.18.x	185.193.44.21	Externe suspecte
Horaire	08h–20h	22h14	Hors plage
MFA	MFA_OK	MFA_FAIL puis BYPASS	Contournement
Durée de l'extraction	Quelques min	< 50 secondes	Automatisé

4.3 Code source – ficoba_analyzer.py

L'outil développé analyse le fichier de log FICOBA, détecte les anomalies selon plusieurs critères, et exporte les alertes dans un fichier CSV pour exploitation SOC.

main.py

mission-3 > main.py

```python
import re
from datetime import datetime
from collections import defaultdict
import csv

internal_networks = ["10.", "172.18."]

def is_internal_ip(ip):
    return any(ip.startswith(net) for net in internal_networks)

log_pattern = re.compile(
    r"\[(?P<timestamp>[\d\-T:]+Z)\]\s*"
    r"(?P<user>[^|]+)\s*\|\s*(?P<role>[^|]+)\s*\|\s*"
    r"(?P<ip>[^|]+)\s*\|\s*(?P<app>[^|]+)\s*\|\s*"
    r"(?P<action>[^|]+)\s*\|\s*(?P<resource>[^|]+)\s*\|\s*"
    r"(?P<query_count>[^|]+)\s*\|\s*(?P<status>[^|]+)\s*\|\s*"
    r"(?P<mfa>[^|]+)\s*\|\s*(?P<session_id>[^|\s]+)"
)

def parse_line(line):
    match = log_pattern.match(line)
    if not match:
        return None
    d = match.groupdict()
    try:
        d["timestamp"] = datetime.fromisoformat(d["timestamp"].replace('Z', '+00:00'))
        d["query_count"] = int(d["query_count"]) if d["query_count"].isdigit() else 0
    except:
        return None
    return d
```

main.py

mission-3 > main.py

```python
def analyze_logs(file_path):
    sessions = defaultdict(list)
    anomalies = []

    with open(file_path, "r") as f:
        text = f.read().replace("\n", " ")  # fusionne tout pour gerer multi-lignes

    for log_text in re.findall(r"\[[\d\-T:]+Z\][^\[]*", text):
        log = parse_line(log_text.strip())
        if not log:
            continue
        sessions[log["session_id"]].append(log)

        if not is_internal_ip(log["ip"]):
            anomalies.append((log, "IP externe suspecte"))
        if log["mfa"] in ["MFA_FAIL", "MFA_BYPASS"]:
            anomalies.append((log, "Probleme MFA"))
        if log["timestamp"].hour < 6 or log["timestamp"].hour > 20:
            anomalies.append((log, "Acces hors horaires"))

    for session_id, logs in sessions.items():
        logs_sorted = sorted(logs, key=lambda x: x["timestamp"])
        for i in range(len(logs_sorted)):
            cumulative = logs_sorted[i]["query_count"]
            start_time = logs_sorted[i]["timestamp"]
            for j in range(i+1, len(logs_sorted)):
                diff = (logs_sorted[j]["timestamp"] - start_time).total_seconds()
                if diff > 10:
                    break
                cumulative += logs_sorted[j]["query_count"]
            if cumulative > 50:
                anomalies.append((logs_sorted[i], f"Rafale rapide ou volume eleve ({cumulative} requêtes)"))
        for log in logs_sorted:
            if log["action"] == "EXPORT" and log["query_count"] > 1000:
                anomalies.append((log, f"Extraction massive ({log['query_count']} lignes)"))

    return anomalies

# --- Analyse ---
file_path = "ficoba_logs.txt"
alerts = analyze_logs(file_path)
```

main.py

mission-3 > main.py

```python
# --- Export CSV ---
with open("ficoba_alertes.csv", "w", newline="") as csvfile:
    writer = csv.writer(csvfile)
    writer.writerow(["Timestamp", "User", "IP", "Action", "Query_Count", "Session_ID", "Anomalie"])
    for log, reason in alerts:
        writer.writerow([log["timestamp"], log["user"], log["ip"], log["action"], log["query_count"], log["session_id"], reason])

print(f"Analyse terminee : {len(alerts)} alertes detectees. CSV 'ficoba_alertes.csv' genere.")
```

4.4 Explication du code

Parsing des logs FICOBA

Le format des logs FICOBA est plus structuré que les logs SSH : chaque champ est séparé par des pipes (|). Une expression régulière avec groupes nommés extrait tous les champs en une seule passe. La fonction parse_line() encapsule ce parsing dans un try/except pour ignorer silencieusement les lignes malformées, conformément aux exigences de développement sécurisé.

Détection multi-critères

L'analyse repose sur quatre détecteurs indépendants appliqués à chaque ligne et à chaque session :

- IP externe : toute IP ne commençant pas par '10.' ou '172.18.' est signalée comme suspecte.
- MFA : les valeurs MFA_FAIL et MFA_BYPASS déclenchent immédiatement une alerte.
- Horaires : les accès entre 20h00 et 06h00 sont considérés hors plage bureautique.
- Rafales : une fenêtre glissante de 10 secondes cumule les query_count ; si le total dépasse 50, une alerte est générée.
- Extraction massive : tout EXPORT avec un query_count supérieur à 1000 lignes déclenche une alerte spécifique.

Gestion des exceptions et sécurité

Contrairement à la Mission 1, la fonction analyze_logs() gère explicitement les cas d'erreur à l'ouverture du fichier (FileNotFoundError, PermissionError) et retourne une liste vide plutôt que de planter. Le parsing ligne par ligne est également protégé. Ces choix sont conformes aux critères de développement sécurisé définis en Mission 2.

Export CSV pour exploitation SOC

Les alertes générées sont exportées dans un fichier CSV (ficoba_alertes.csv) contenant pour chaque alerte : le timestamp, l'utilisateur, l'IP, l'action, le volume de requêtes, l'identifiant de session et le type d'anomalie détectée. Ce format est directement importable dans un SIEM ou un outil d'analyse SOC.

4.5 Résultats

A1 | Timestamp,User,IP,Action,Query_Count,Session_ID,Anomalie

	A	B	C	D	E	F	G	H
1	Timestamp,User,IP,Action,Query_Count,Session_ID,Anomalie							
2	2026-01-20 08:00:46+00:00,p.leroy ,163.226.162.17 ,SEARCH ,0,SID591236,IP externe suspecte							
3	2026-01-20 08:05:13+00:00,t.simon ,190.212.219.128 ,LOGOUT ,0,SID151369,IP externe suspecte							
4	2026-01-20 08:07:20+00:00,n.chevalier ,101.67.95.213 ,LOGIN ,0,SID232035,IP externe suspecte							
5	2026-01-20 08:14:17+00:00,n.chevalier ,121.28.135.54 ,LOGIN ,0,SID472255,IP externe suspecte							
6	2026-01-20 08:23:31+00:00,f.laurent ,163.197.123.49 ,EXPORT ,0,SID925507,IP externe suspecte							
7	2026-01-20 08:25:02+00:00,e.boyer ,184.107.148.136 ,EXPORT ,0,SID584899,IP externe suspecte							
8	2026-01-20 08:31:00+00:00,j.dupont ,134.44.199.30 ,SEARCH ,0,SID448911,IP externe suspecte							
9	2026-01-20 08:43:43+00:00,p.leroy ,86.38.52.148 ,LOGOUT ,0,SID564458,IP externe suspecte							
10	2026-01-20 09:00:21+00:00,m.bernard ,103.230.137.209 ,SEARCH ,0,SID838535,IP externe suspecte							
11	2026-01-20 09:04:02+00:00,s.durand ,132.25.167.155 ,LOGIN ,0,SID955730,IP externe suspecte							
12	2026-01-20 09:15:13+00:00,m.bernard ,188.19.145.107 ,LOGOUT ,0,SID354098,IP externe suspecte							
13	2026-01-20 09:28:41+00:00,p.leroy ,184.93.123.141 ,LOGIN ,0,SID820749,IP externe suspecte							
14	2026-01-20 10:12:02+00:00,d.michel ,198.206.113.50 ,SEARCH ,0,SID135803,IP externe suspecte							
15	2026-01-20 10:19:13+00:00,r.garnier ,80.235.22.100 ,LOGIN ,0,SID384077,IP externe suspecte							
16	2026-01-20 10:22:15+00:00,f.laurent ,87.138.75.107 ,LOGOUT ,0,SID458451,IP externe suspecte							
17	2026-01-20 10:23:36+00:00,f.laurent ,89.112.4.63 ,LOGOUT ,0,SID734427,IP externe suspecte							
18	2026-01-20 10:30:19+00:00,s.durand ,101.42.222.179 ,LOGOUT ,0,SID791549,IP externe suspecte							
19	2026-01-20 10:31:51+00:00,d.michel ,139.41.104.211 ,LOGIN ,0,SID574326,IP externe suspecte							
20	2026-01-20 10:34:53+00:00,d.michel ,103.217.84.49 ,LOGOUT ,0,SID138710,IP externe suspecte							
21	2026-01-20 10:35:07+00:00,p.leroy ,97.104.192.148 ,SEARCH ,0,SID185704,IP externe suspecte							
22	2026-01-20 10:35:17+00:00,p.leroy ,135.24.4.227 ,EXPORT ,0,SID966082,IP externe suspecte							

5. Conclusion

Ce TP a permis de construire progressivement trois outils d'analyse de sécurité en Python, couvrant des problématiques complémentaires : la détection d'attaques réseau sur SSH, la mise en conformité avec les exigences de développement sécurisé, et l'analyse comportementale applicative dans un contexte SOC réel.

La Mission 1 a validé l'approche algorithmique de base : grâce aux expressions régulières, aux structures de données adaptées et à la détection par fenêtre temporelle glissante, le programme identifie efficacement les comportements suspects. Sur le fichier fourni, près de 35 000 tentatives échouées ont été détectées, avec deux IPs responsables d'attaques de brute force dépassant 400 tentatives par minute.

La Mission 2 a mis en évidence les limites d'un code conçu pour fonctionner mais pas pour résister à des conditions réelles de production. La refonte en modules distincts, combinée à la gestion des exceptions, à la protection contre le log flooding, à la validation des données et au mode temps réel, transforme le script en un outil fiable et maintenable, exploitable directement par une équipe SOC.

La Mission 3 illustre comment ces mêmes principes s'appliquent à un contexte applicatif sensible. L'analyse des logs FICOBA montre qu'aucun indicateur isolé ne suffit à détecter une compromission : c'est la corrélation entre une IP externe, un contournement MFA, un volume d'export massif et un horaire atypique qui permet de caractériser l'attaque. L'export CSV des alertes constitue un livrable directement exploitable dans un SIEM.

Ces trois missions illustrent concrètement un principe fondamental du développement sécurisé : un outil de sécurité n'est fiable que s'il est lui-même développé de manière sécurisée — robuste face aux erreurs, résistant aux données malformées, et conçu pour s'intégrer dans des processus opérationnels réels.